

HESTIA

Documentação C4 do HestIA

Contexto, contêineres, componentes, código, cenário dinâmico e aderência ao repositório

Tarefa: SCRUM-105

Disciplina: Modelagem e Construção de Software

Épico Jira: SCRUM-99 - USD02

Sprint: Sprint 1

Responsável no Jira: Ricardo Gul D'Avila

Data-base: 2 de setembro de 2026

Status do documento: Versão de trabalho para revisão da equipe

Base de elaboração: materiais do Canvas, estudo de caso do PI-IV, cards do Jira, ADR-001 e estado observado do repositório HestIA. Os rótulos de disciplina do Jira foram tratados como metadados, não como fonte acadêmica principal.

1. Objetivo e convenções

O modelo C4 documenta a arquitetura em níveis progressivos de zoom: Contexto, Contêiner, Componente e Código. Esta documentação representa a arquitetura-alvo do MVP definida na SCRUM-106 e identifica explicitamente elementos ainda não implementados.

Notação	Significado
Caixa azul/verde	Elemento interno do HestIA ou do MVP.
Caixa cinza	Pessoa, banco ou elemento de infraestrutura.
Caixa amarela/roxa	Integração auxiliar ou futura.
Seta contínua	Relação necessária no MVP.
Seta tracejada	Relação planejada, ainda não implementada.

2. Nível 1 - Contexto do Sistema

C4 - Nível 1: Contexto do Sistema

Pessoas e sistemas que interagem com o HestIA

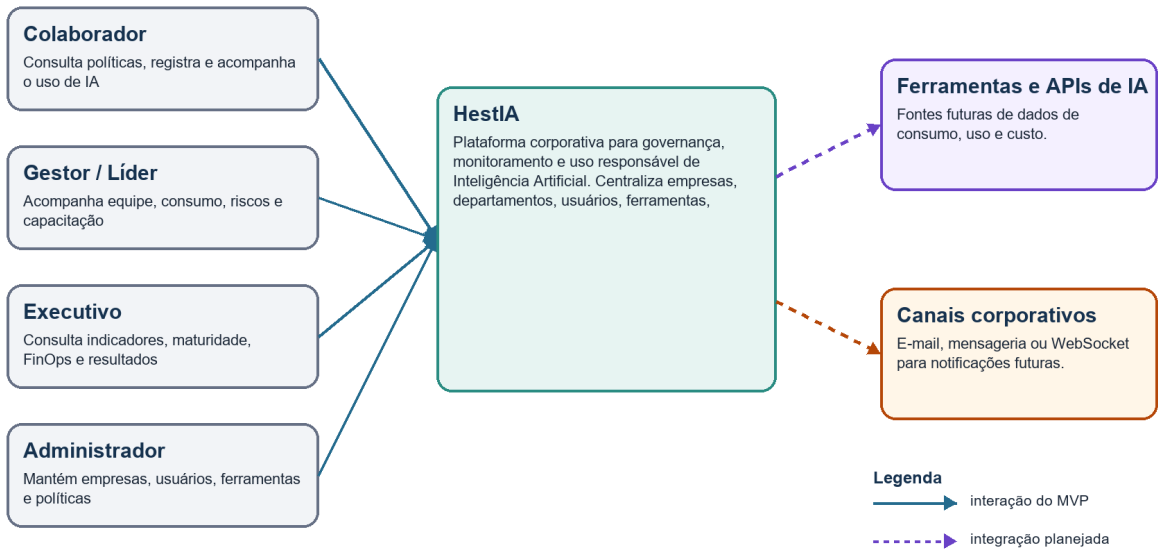


Figura 1 - Diagrama de Contexto do HestIA.

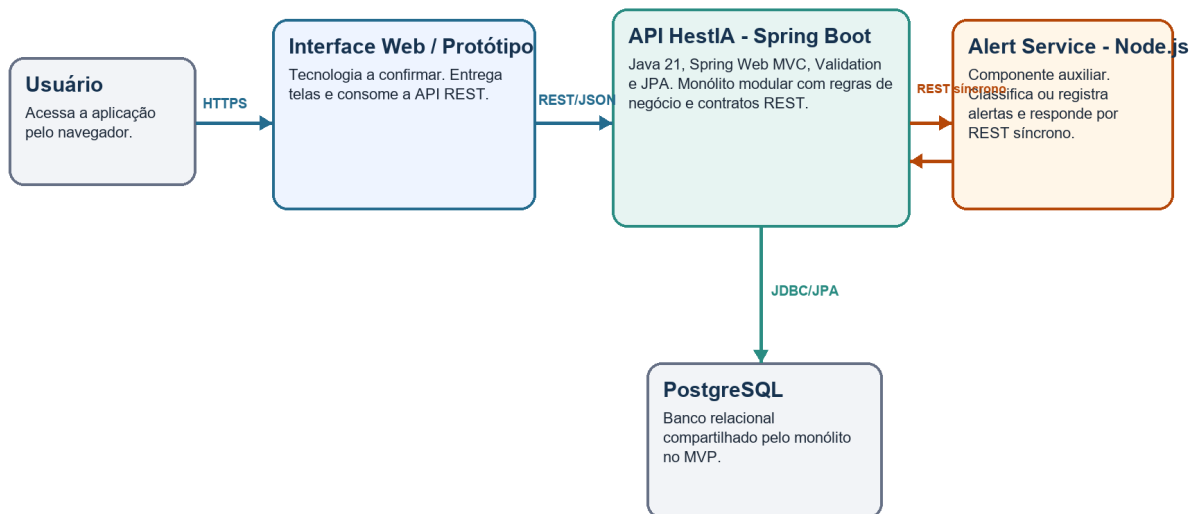
Elemento	Tipo	Responsabilidade / relação
HestIA	Sistema	Centraliza governança, uso, risco, políticas, ferramentas e indicadores de IA.
Colaborador	Pessoa	Consulta políticas e registra/acompanha uso e capacitação.
Gestor/Líder	Pessoa	Acompanha equipe, alertas, consumo e evolução.
Executivo	Pessoa	Consulta maturidade, produtividade, risco e FinOps.
Administrador	Pessoa	Mantém cadastros, permissões, ferramentas e políticas.

Elemento	Tipo	Responsabilidade / relação
Ferramentas/APIs de IA	Sistema externo futuro	Fornecer telemetria ou consumo quando integrações forem implementadas.
Canais corporativos	Sistema externo futuro	Receber notificações por e-mail, mensageria ou canal em tempo real.

3. Nível 2 - Contêineres

C4 - Nível 2: Contêineres

Arquitetura-alvo do MVP e estado de cada contêiner



Implantação proposta: Docker Compose com serviços hestia-api, postgres e alert-service; interface web adicionada quando o protótipo for versionado.
O Alert Service não possui autonomia de domínio suficiente para alterar a decisão principal: o núcleo continua sendo um monólito modular.

Figura 2 - Diagrama de Contêineres do MVP.

Contêiner	Tecnologia	Responsabilidade	Estado
Interface Web / Protótipo	A confirmar	Interface para os fluxos demonstráveis.	Planejado; não encontrado no repositório.
API HestIA	Java 21, Spring Boot	Casos de uso, regras, APIs e persistência do núcleo.	Parcialmente implementado.
PostgreSQL	PostgreSQL	Dados corporativos do monólito modular.	Dependência definida; configuração a consolidar.
Alert Service	Node.js / Express	Avalia ou registra alertas e devolve nível/mensagem.	Planejado pelas SCRUM-109/159.

Decisão sobre Node.js: o Alert Service é um componente auxiliar integrado por REST síncrono, conforme o exemplo acadêmico. Ele não altera, por si só, o estilo do núcleo para microsserviços.

4. Nível 3 - Componentes da API HestIA

C4 - Nível 3: Componentes da API HestIA

Fronteiras do monólito modular e integração externa

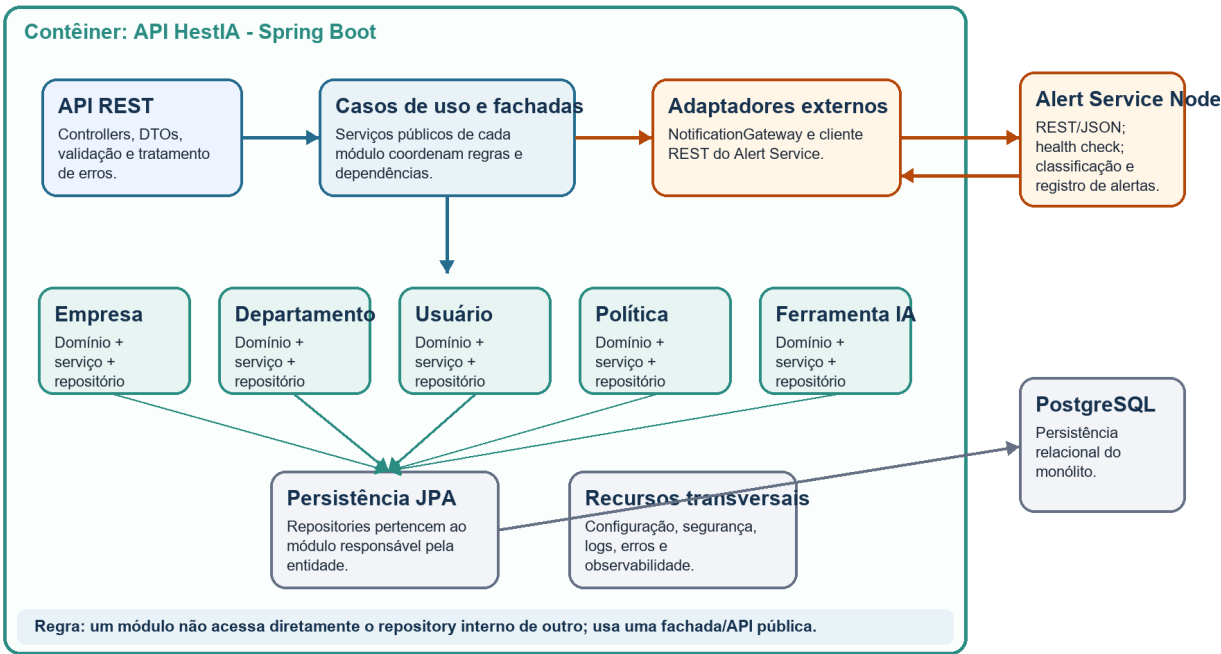


Figura 3 - Componentes e limites do monólito modular.

Componente	Responsabilidade	Dependências permitidas
API REST	Receber HTTP, validar DTOs e traduzir respostas/erros.	Casos de uso e fachadas públicas.
Empresa	Empresa, orçamento e configuração corporativa.	Persistência própria.
Departamento	Estrutura organizacional.	EmpresaFacade, nunca EmpresaRepository direto.
Usuário	Usuários, perfis e vínculo organizacional.	EmpresaFacade e DepartamentoFacade.
Política	Políticas, versões e ativação.	EmpresaFacade.
Ferramenta IA	Catálogo, aprovação, risco e custo.	EmpresaFacade.
Consumo/Indicadores	Registros de uso e agregações iniciais.	Ferramenta, empresa e NotificationGateway.
NotificationGateway	Porta de integração com o Alert Service.	Cliente REST externo; sem regra do domínio Node.
Persistência JPA	Adapters de banco pertencentes a cada módulo.	PostgreSQL.

5. Nível 4 - Visão de código

O nível de código é apresentado como estrutura recomendada de pacotes. Classes concretas devem acompanhar a evolução do repositório; o documento não pretende substituir diagramas de classe detalhados.

```
br.com.hestia +-+ HestiaApplication +-+ empresa | +-+ api/EmpresaController, EmpresaRequest, EmpresaResponse | +-+
application/EmpresaFacade, EmpresaService | +-+ domain/Empresa | +-+ infrastructure/EmpresaRepository +-+ departamento +-+
usuario +-+ politica +-+ ferramenta +-+ consumo +-+ notificacao | +-+ application/NotificationGateway | +-+
infrastructure/NodeAlertClient +-+ shared/config | error | security | observability
```

6. Cenário dinâmico - registro de consumo e alerta

1. O usuário registra um uso/consumo na Interface Web.
2. A Interface Web envia POST para a API HestIA.
3. A API valida usuário, empresa, ferramenta e política aplicável.
4. O módulo Consumo persiste o registro no PostgreSQL.
5. O NotificationGateway envia ao Alert Service os dados necessários para avaliação.
6. O Alert Service responde com nível INFO, WARNING ou CRITICAL e uma mensagem.
7. A API devolve a resposta consolidada à Interface Web.
8. Em timeout ou indisponibilidade, o consumo permanece válido e o alerta fica pendente ou é reportado como não avaliado.

Chamada	Origem -> destino	Contrato resumido
POST /api/v1/consumos	Interface -> API HestIA	empresald, usuariold, ferramentald, quantidade, unidade e contexto.
POST /alerts/evaluate	API HestIA -> Alert Service	empresald, origem, métrica, valor, limites e correlationId.
200 AlertResponse	Alert Service -> API HestIA	level, message, evaluatedAt e correlationId.

7. Implantação

```
docker-compose +-+ postgres :5432 [health: pg_isready] +-+ hestia-api :8080 [depende do postgres saudável] +-+ alert-service
:3001 [health: /health] +-+ web-ui :3000 [planejado; depende da API]
```

- URLs, portas e credenciais são fornecidas por variáveis de ambiente.
- O banco não deve publicar credenciais no repositório.
- Health checks determinam a ordem de inicialização.
- Logs da API e do Node compartilham correlationId para rastrear o fluxo.

8. Coerência com o código e lacunas

Evidência do repositório	Representação no C4	Lacuna
Pacotes empresa, departamento, usuario e politica	Módulos dentro da API HestIA.	Criar contratos públicos/fachadas.
TipoFerramentaIA	Módulo Ferramenta IA.	Entidade e camadas restantes pendentes.
Duas entidades PolíticaUso	Um único componente Política.	Remover duplicidade antes do build final.
Services acessando repository de outro módulo	Setas passam por fachadas.	Refatorar dependências diretas.
Ausência de Node	Alert Service marcado como planejado.	Implementar nas Scrums seguintes.
Ausência de frontend versionado	Interface marcada como planejada.	Versionar protótipo ou documentar ferramenta externa.

9. Critérios de aceite da SCRUM-105

1. A documentação contém Contexto, Contêineres, Componentes e visão de Código.
2. Cada elemento possui nome, tipo, responsabilidade, tecnologia e relações compreensíveis.
3. Os diagramas distinguem estado implementado, planejado e futuro.
4. As relações respeitam a ADR-001 e as regras da SCRUM-106.
5. O cenário Spring Boot -> Node.js -> resposta está documentado.
6. As lacunas entre diagrama e repositório estão registradas para correção.

Referências e fontes consultadas

- Canvas ESPM. Deliverables 1 (Entregáveis 1) - Todas as Disciplinas. Curso GTECHSP-TI4A-2592, atividade 293213.
- LELES, Andréia Damasio de. Introdução à Arquitetura de Software - C4_v1. Material da disciplina Modelagem e Construção de Software, 2026.
- LELES, Andréia Damasio de. Monólitos em Camadas x Monólito Modular x Microservices. Material da disciplina, 2026.
- LELES, Andréia Damasio de. Monólitos em camadas e microservices: integração com Node via RestClient. Material da disciplina, 2026.
- ESPM. PI-IV - Projeto Estudo de Caso v2. Projeto Integrado IV, 2026.
- HestIA. ADR-001 - Adotar monólito modular com camadas internas por módulo de negócio. Status: aceita, 30 ago. 2026.
- Jira Projeto_SM. SCRUM-105, SCRUM-106 e SCRUM-150. Consultado em 2 set. 2026.
- Repositório local HestIA. Snapshot técnico de origin/develop e branch local de documentação, consultado em 2 set. 2026.
- BROWN, Simon. The C4 Model for Visualising Software Architecture. c4model.com.